

GALXY

Custom Lighting & Craft Studio

Module 5 — AI Preview Generation

AI Provider Integration · Prompt Templating · Caching & Rate Limiting
Consumer of Module 2 (Categories) & Module 4 (Configurator/Pricing)

Prepared by LTI Technology | Learn · Think · Innovate

Stack: Flask · MongoDB · Cloudinary · Gemini (free-tier)

Document Date: July 2026

1. Scope

This module generates an AI-rendered preview image of a customer's custom configuration before they commit to an order — turning abstract attribute selections ("Cursive font, Blue neon, with chain") into an actual visual they can see. It owns the AI provider integration, per-category prompt templating, generation history and logging, and cost-control mechanisms (caching, rate limiting), since the AI provider's free tier has real usage limits.

This module is a consumer of Module 2 (category schema + prompt template) and Module 4 (attribute validation) — it never accepts unvalidated input and never builds a category's prompt logic itself.

2. Collection Owned by This Module

2.1 ai_generations

```
{
  "_id": ObjectId,
  "user_id": "ObjectId or null (guests can use AI preview without login)",
  "session_id": "string — for guest tracking / rate limiting before signup",
  "category_id": ObjectId,
  "product_id": "ObjectId or null (null if fully custom, no base product)",
  "selected_attributes": {},
  "prompt_used": "string — final assembled prompt sent to the provider",
  "input_reference_image": "cloudinary_url or null",
  "output_image_url": "cloudinary_url",
  "provider": "gemini",
  "status": "success | failed | rate_limited",
  "error_message": "string or null",
  "generation_time_ms": 0,
  "created_at": "datetime"
}
```

Indexes

- user_id — for fetching a user's generation history
- session_id — for guest rate limiting
- category_id — for admin usage analytics per category
- created_at — for cleanup/expiry jobs and recent-activity queries

Note: guests (not logged in) are allowed to use AI preview — this is a key conversion tool; gating it behind signup would kill trial usage. Track guests via session_id (a random UUID set in a cookie/localStorage by the frontend on first visit) instead of user_id.

3. Folder Structure (Flask)

```
app/
models/
```

```

ai_generation.py
services/
  ai_service.py          - orchestrates: validate → build prompt → call provider → upload
                           result → log
  prompt_builder_service.py - fills category's ai_prompt_template with selected_attributes
  ai_provider_client.py   - thin wrapper around the Gemini API call itself
  ai_cache_service.py     - checks/stores cached results for identical attribute combos
  ai_rate_limit_service.py - enforces per-session/per-user generation limits
routes/
  ai_routes.py
  admin_ai_routes.py
configs/
  ai_config.py           - GEMINI_API_KEY, provider selection, rate limit thresholds

```

4. Environment Variables

```

AI_PROVIDER=gemini
GEMINI_API_KEY=
AI_FREE_GENERATIONS_PER_SESSION=5
AI_MAX_GENERATIONS_PER_USER_PER_DAY=20
AI_GENERATION_TIMEOUT_SECONDS=30

```

AI_PROVIDER is read at runtime by ai_provider_client.py to pick which SDK/API call to make — keep this genuinely swappable (an interface/adaptor pattern with one function per provider), since free-tier limits or quality may force a provider switch later without a full rewrite.

5. Prompt Building — prompt_builder_service.py

Input: category.ai_prompt_template (from Module 2) + selected_attributes (already validated by Module 4) + optional customer free-text (e.g. the name/word to render, if the category has a text_input attribute).

Template example (stored per-category in MongoDB, editable by Asil)

```

A realistic professional product photo of a custom {category_name}
spelling "{custom_text}", in {color} color, {font} font style,
{chain} hanging chain, mounted on a dark background with soft
ambient glow, studio lighting, high detail, no watermark, no text overlay

```

Placeholder resolution rules

- Placeholders map directly to attribute.key values from selected_attributes — e.g. {font} pulls the selected option's label (not raw value), so the prompt reads naturally ("Bold" not "bold_v2").
- Attributes with "affects_ai_preview": false in their schema are excluded from prompt-building entirely, even if present in selected_attributes — this keeps prompts focused (e.g. an internal SKU-style attribute shouldn't leak into image generation text).
- {custom_text} (or similarly flagged text_input fields) should be sanitized before insertion — strip anything resembling prompt-injection attempts ("ignore previous instructions", excessive special characters) since this is the one truly free-text field a customer controls.

- If a placeholder in the template has no corresponding selected attribute (e.g. optional attribute left unset), replace it with an empty string and clean up resulting double-spaces/punctuation, rather than leaving literal {key} text in the final prompt.

This function's output (prompt_used) is what gets logged in ai_generations — always store the exact final prompt, never just the raw attributes, so Asil can debug why a particular generation looked the way it did.

6. Generation Flow — ai_service.py

```
1. Receive request: category_id, product_id (optional), selected_attributes, session_id/user_id
2. Call Module 4's validate_attributes(category, selected_attributes)
   → if invalid, return 400 immediately, no AI call made (never waste a
     generation on a broken selection)
3. Check ai_rate_limit_service — has this session/user exceeded their quota?
   → if yes, return 429 with a clear message, no AI call made
4. Check ai_cache_service — has this exact (category_id + selected_attributes)
   combination been generated successfully before?
   → if yes, return the cached output_image_url immediately, log a new
     ai_generations record with status "success" but generation_time_ms: 0
     and a flag noting it was served from cache (add "from_cache": true to
     the response, not stored on the model — informational only)
5. Build prompt via prompt_builder_service
6. Call ai_provider_client → Gemini API
   → timeout enforced (AI_GENERATION_TIMEOUT_SECONDS)
7. On success: upload returned image to Cloudinary, log ai_generations
   with status "success", return output_image_url
8. On failure/timeout: log ai_generations with status "failed" +
   error_message, return a clean error to the frontend (never expose raw
   provider error text/API details to the client)
```

7. Caching Strategy — ai_cache_service.py

Cache key = deterministic hash of category_id + sorted(selected_attributes). On an identical combination:

- Return the previously generated output_image_url instantly.
- This is essential for free-tier cost control — many customers will land on the same product page and leave attributes at their defaults, generating identical prompts repeatedly without caching.

Cache is not per-user — if two different customers select the exact same neon sign configuration (same font/color/chain, generic no-name-yet preview), they can share a cached result. Only bypass cache when custom_text (or any customer-specific free text) is part of selected_attributes, since that makes each generation unique by definition — never cache across different custom_text values.

8. Rate Limiting — ai_rate_limit_service.py

Two tiers, both configurable via `ai_config.py`:

- Guest (session-based): `AI_FREE_GENERATIONS_PER_SESSION` — once exceeded, frontend should prompt "Sign up to keep designing" rather than a dead-end error.
- Logged-in user (daily): `AI_MAX_GENERATIONS_PER_USER_PER_DAY` — higher ceiling than guest, resets daily.

Cached hits (Section 7) should not count against rate limits — only genuine new provider calls consume quota. This matters: a customer re-visiting the same default configuration shouldn't be penalized.

9. API Contract

POST `/api/ai/generate-preview`

Body: { "category_id", "product_id?", "selected_attributes": {}, "session_id" }
Auth: optional (works for guest via `session_id`, or logged-in user)

Success 200:

```
{ "success": true, "data": {  
  "output_image_url", "from_cache": false,  
  "disclaimer": "AI-generated approximation – final product may vary."  
} }
```

Errors:

400 – invalid attributes (same errors shape as Module 4)
429 – rate limit exceeded: { "success": false, "message": "...", "data": { "limit_reached": true } }
504 – provider timeout
502 – provider error (generic message, no internal details leaked)

GET `/api/ai/generations/:user_id`

Auth required, must match `request.user_id` (or admin)
Returns this user's generation history, most recent first, paginated.
Used for a "your recent designs" panel so customers can revisit/reuse past previews without regenerating.

GET `/api/admin/ai/generations`

Admin only. Full list, filterable by category/status/date, paginated.
Powers the admin AI-usage analytics view (Module 12 dashboard pulls aggregate stats from here).

PUT `/api/admin/categories/:id/ai-prompt-template`

Body: { "ai_prompt_template": "string" }

Note: this route technically modifies the categories collection (Module 2's data), so implement it there OR here — but only one module should own the write, to avoid duplicate logic. Recommendation: Module 2 owns the route (it already owns category writes), Module 5 only reads the template. Flag this ownership decision explicitly between the two TLs to avoid duplicate endpoints being built.

10. Cloudinary Integration Note

Reuse the shared Cloudinary upload helper (flag with Module 3/10 to consolidate into one `app/utils/cloudinary_helper.py` rather than each module writing its own wrapper). AI-generated images should be uploaded to a distinct Cloudinary folder (e.g. `galaxy/ai-previews/`) separate from real product photos, so admin media browsing and cleanup jobs can distinguish generated content from actual product photography.

11. What This Module Does NOT Do

- Does not define or edit attribute schemas or prompt templates (Module 2 owns category writes — this module only reads `ai_prompt_template`).
- Does not validate attribute selections itself — always delegates to Module 4, never re-implements validation rules.
- Does not calculate price — AI preview is visual only, price comes from Module 4 independently.
- Does not persist the preview into a cart item automatically — Module 6 (Cart) is responsible for storing `ai_preview_image` on a cart item when the customer adds to cart; this module just returns the URL for the frontend to hold onto until then.

12. Handoff Contract for Other Teams

Treat as frozen from Day 1:

Contract Item	Detail
Response shape	POST <code>/api/ai/generate-preview</code> response always includes <code>output_image_url</code> and disclaimer — Module 6 (Cart) copies <code>output_image_url</code> verbatim into <code>cart_item.ai_preview_image</code> when a customer adds an item; it does not re-generate or re-fetch.
Rate-limit shape	429 with <code>limit_reached: true</code> — the frontend AI Preview Panel component needs this exact shape to show the correct upsell/wait messaging per the design system's loading-state rules.
URL stability	<code>ai_generations.output_image_url</code> is a permanent, stable Cloudinary URL — safe for Module 8 (Orders) to store a copy of this URL directly on the order line item (<code>items[].ai_preview_image</code>) at checkout time, since it won't be deleted or change later.